PROJECT REPORT

GLASS DETECTION USING CNN(MODULE 4)

SUBMITTED BY

MEGHA L NARAYANAN

INTRODUCTION

The rapid advancement of Artificial Intelligence and Deep Learning has significantly improved the ability of machines to analyse and interpret visual data. A key area within this domain is Computer Vision, which focuses on enabling systems to understand images and videos.

One of the most powerful techniques used in computer vision is the Convolutional Neural Networks. CNNs are designed to automatically extract important features from images, eliminating the need for manual feature engineering and improving classification accuracy.

In this project, a CNN-based model is developed to classify images into two categories: **Glasses and No Glasses**. The model is trained on a dataset of labelled images and learns to identify visual patterns associated with eyewear under different conditions.

To enhance usability, the trained model is deployed using Streamlit, allowing users to upload images and obtain predictions along with confidence scores. This project demonstrates the practical application of deep learning in solving real-world image classification problems.

DATASET DESCRIPTION

The dataset used in this project consists of facial images categorized into two classes: Glasses and No Glasses. It is designed for a binary image classification task, where the goal is to train a model to distinguish whether a person is wearing glasses or not.

Dataset Structure

The dataset is organized into separate folders for each class:

- Glasses → Images of individuals wearing glasses
- No Glasses → Images of individuals not wearing glasses

This folder-based structure allows easy loading using data generators during model training.

1. CODE IMPLEMENTATION AND EXPLANATION

1.1Import required libraries

```
#!pip install opencv-python
```

```
import pandas as pd
import numpy as np
import os
import cv2
import tensorflow as tf
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix,classification_report
from sklearn.feature_extraction.text import CountVectorizer,TfidfVectorizer
import warnings
warnings.filterwarnings("ignore")
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D,MaxPool2D,Dense,Flatten
```

This code initializes the environment for a machine learning and deep learning project by importing all the necessary libraries. It includes pandas and numpy for data handling and numerical computations, along with os for file management. The cv2 (OpenCV) library is used for image processing tasks, while TensorFlow provides the framework for building and training deep learning models. Visualization is supported using seaborn, which helps in plotting performance metrics. Several modules from scikit-learn are imported to split the dataset, encode labels, evaluate model performance using confusion matrix and classification reports, and convert textual data into numerical form using techniques like CountVectorizer and TF-IDF. Additionally, warnings are suppressed to keep the output clean. Finally, specific components from TensorFlow's Keras API, such as Sequential, Conv2D, MaxPool2D, Flatten, and Dense layers, are imported to construct a Convolutional Neural Network (CNN) for tasks like image classification.

1.2Load the dataset

```
data1=r"C:\Users\HP\Desktop\datascience notes\deep learning datasets\archive (2)\glasses-noglasses\train"
```

Loaded the dataset as data1.

1.3 Convert to a format to train in CNN

```
x,y=[],[]
for i in os.listdir(data1):
    new_dir=os.path.join(data1,i)
    if os.path.isdir(new_dir):
        for j in os.listdir(new_dir):
            image_path=os.path.join(new_dir,j)
            if image_path.endswith((".png",".jpg",".jpeg")):
                image_data=cv2.imread(image_path)
                array_resize=cv2.resize(image_data,(100,100))
                array_reshape=array_resize.reshape(100,100,3)
                array_norm=array_reshape/255
                x.append(array_norm)
                y.append(i)
x=np.array(x1)
y=np.array(y1)
print(x.shape)
print(y.shape)
```

```
104, 100, 100, 3)
104,)
```

This code is used to load and preprocess image data for a CNN model by reading images from folders and converting them into numerical arrays.

It starts by creating two empty lists, x and y, where x will store the processed image data and y will store the corresponding labels. The loop goes through each folder inside data1 using os.listdir(). Each folder name (i) typically represents a class label (for example, “Pneumonia” or “Normal”). For every folder, it checks if it is a directory and then loops through all the image files inside it.

For each image, the code constructs its full path and checks whether the file using cv2.imread(), then resized to 100×100 pixels using cv2.resize() to ensure

all images have the same dimensions. The resized image is reshaped into (100, 100, 3) to match RGB format, and then normalized by dividing pixel values by 255 so they fall between 0 and 1, which helps improve model performance.

Each processed image is appended to x, and its corresponding label (folder name i) is appended to y. After processing all images, both lists are converted into NumPy arrays. Finally, the shapes are printed:

- x.shape = (104, 100, 100, 3) → 104 images, each of size 100×100 with 3 color channels
- y.shape = (104,) → 104 labels corresponding to each image

1.4 Convert Y to numerical values

```
le=LabelEncoder()
y=le.fit_transform(y)
y
```

Label encoder is used to convert the categorical values in variable y to numerical values

1.5 Convolutional Neural Network (CNN) Architecture

```
model=Sequential()
model.add(Conv2D(32,(3,3),activation="relu",input_shape=(100,100,3)))
model.add(MaxPool2D(2,2))
model.add(Conv2D(64,(3,3),activation="relu"))
model.add(MaxPool2D(2,2))
model.add(Conv2D(128,(3,3),activation="relu"))
model.add(MaxPool2D(2,2))
model.add(Flatten())
model.add(Dense(128,activation="relu"))
model.add(Dense(5,activation="softmax"))
```

This code defines a Convolutional Neural Network (CNN) model using Keras' Sequential API for image classification.

It begins by creating a sequential model, where layers are added one after another. The first layer is a Conv2D layer with 32 filters and a 3×3 kernel, which extracts basic features like edges and textures from the input images of size 100×100×3. This is followed by a MaxPooling layer (2×2) that reduces the spatial size of the feature maps, helping to lower computation and retain

important features. The pattern is repeated with deeper layers: a second convolutional layer with 64 filters and a third with 128 filters, each followed by max pooling, allowing the model to learn increasingly complex patterns.

After feature extraction, the Flatten layer converts the 2D feature maps into a 1D vector so it can be fed into fully connected layers. A Dense layer with 128 neurons and ReLU activation is then used to learn high-level representations. Finally, the output layer is a Dense layer with 5 neurons and softmax activation, which produces probability scores for 5 different classes, making this a multi-class classification model.

1.6 Training the model

```
model.compile(optimizer="adam",loss="sparse_categorical_crossentropy",metrics=["accuracy"])
```

The model.compile() line sets up how the model will learn. The Adam optimizer is used to adjust the model weights efficiently during training. The loss function sparse_categorical_crossentropy is chosen because your labels are in integer form (not one-hot encoded), and it measures how well the predicted class probabilities match the actual classes. The metric accuracy is included to track how many predictions the model gets correct during training.

```
model.fit(xtrain,ytrain,batch_size=10,epochs=30,validation_data=(xtest,ytest))
```

The model.fit() line actually trains the model using your data. Here, xtrain and ytrain are the training images and labels. The model trains for **30 epochs**, meaning it goes through the entire dataset 30 times, and uses a **batch size of 10**, which means it processes 10 samples at a time before updating weights. The validation_data=(xtest, ytest) part allows the model to evaluate its performance on unseen test data after each epoch, helping you monitor whether the model is learning well or overfitting.

1.7 Testing the trained labels

```
test_data1=r"C:\Users\HP\Desktop\datascience notes\deep learning datasets\archive (2)\glasses-noglasses\train\glasses\row-2-column-4.jpg"

image_data=cv2.imread(test_data1)
array_resize=cv2.resize(image_data,(100,100))
array_reshape=array_resize.reshape(1,100,100,3)
array_norm=array_reshape/255
print(array_norm.shape)

1, 100, 100, 3)

a=model.predict(array_norm)

/1 ━━━━━━━━━━━━━━━━━━━━ 0s 99ms/step

label=["GLASSES....","NO GLASSES..."]

b=np.argmax(a)
label[b]

NO GLASSES...'
```

The code processes an image to determine if someone is wearing glasses.

1. Preprocessing: It loads the image, resizes it to 100x100 pixels, and normalizes pixel values to a 0–1 range so the model can read it.
2. Reshaping: It transforms the data into a 4D shape $(1, 100, 100, 3)$ to satisfy the "batch" format required by deep learning frameworks like TensorFlow or Keras.
3. Prediction: The model analyzes the image and returns probabilities for two categories.
4. Classification: np.argmax picks the highest probability. In your case, it correctly identified the class as "NO GLASSES...".

1.8 Saving the trained model

```
model.save("glasses.model.h5")
```

The trained model is saved as glasses.model as an h5 file

2.Create a streamlit Glasses Detection app

```
from tensorflow.keras import backend as K
import streamlit as st
import numpy as np
import cv2
import tensorflow as tf
from PIL import Image
K.clear_session()

# Load trained model

def load_model():
    model = tf.keras.models.load_model("glasses.model.h5")
    return model

model = load_model()

# App Title
st.title("👓 Glasses Detection App")
st.write("Upload an image to check whether the person is wearing glasses or not.")

# Upload image
uploaded_file = st.file_uploader("Choose an image...", type=["jpg", "png", "jpeg"])

# Class labels
class_names = ["Glasses", "No Glasses"]
```

```
def preprocess_image(uploaded_file):
    file_bytes= np.asarray(bytearray(uploaded_file.read()), dtype=np.uint8)
    image = cv2.imdecode(file_bytes, cv2.IMREAD_COLOR)
    array_resize=cv2.resize(image,(100,100))
    array_reshape=array_resize.reshape(1,100,100,3)
    array_norm=array_reshape/255
    print(array_norm.shape)
    return array_norm

if uploaded_file is not None:
    image = Image.open(uploaded_file)
    st.image(image, caption="Uploaded Image", use_column_width=True)
    uploaded_file.seek(0)
    if st.button("Predict"):
        processed_img = preprocess_image(uploaded_file)
        prediction = model.predict(processed_img)

        prob = prediction[0]
        predicted_class = np.argmax(prob)
        confidence = np.max(prob)

        st.subheader("Prediction:")
        st.write(f"**{class_names[predicted_class]}**")

        st.subheader("Confidence Score:")
        st.write(f"{confidence * 100:.2f}%")
```

1. Environment Setup & Model Loading

imports essential libraries (TensorFlow, Streamlit, and OpenCV for image processing).

- **K.clear_session()**: Clears any existing models from memory to prevent slowdowns.
- **load_model()**: Loads your pre-trained deep learning file (glasses.model.h5), which contains the "knowledge" needed to recognize eyewear.

2. Streamlit User Interface

This part handles how the user interacts with the app in a web browser.

- **st.title & st.file_uploader**: Creates a clean header and a "drag-and-drop" box for users to upload JPG or PNG images.
- **class_names**: Sets the human-readable labels for the final output: "Glasses" or "No Glasses."

3. Image Preprocessing

Decoding: Converts the uploaded raw bytes into an image format OpenCV can understand.

Resizing & Normalizing: Shrinks the image to **100x100** pixels and scales the colors (dividing by 255) to match the model's training requirements.

4. Prediction & Result Display

model.predict: Runs the processed image through the neural network.

np.argmax & np.max: Identifies which category is most likely and calculates the **Confidence Score** (e.g., "98% sure").

Output: Displays the final verdict and the percentage of certainty directly on the webpage.

3.DISPLAYING OUTPUTS

3.1WITH GLASSES

3.2 WITHOUT GLASSES

CONCLUSION

The developed Glasses Detection Web Application successfully integrates a deep learning backend with a user-friendly Streamlit interface. By utilizing Convolutional Neural Networks (CNN) for feature extraction and OpenCV for real-time image preprocessing, the system provides an efficient end-to-end pipeline for binary classification.

The application demonstrates how complex machine learning models can be made accessible through web deployment, allowing users to upload images and receive instant predictions with associated confidence scores. Future iterations could improve accuracy by expanding the training dataset to include diverse lighting conditions and varied eyewear styles, further refining the model's robustness for real-world deployment.